# Lesson Planning

## Complete Lesson Plan Book

**Class: Class 10**

**Subject: Computer Science**

**Medium: English**

Academic Year 2026-27

This book is prepared as a teaching guide for teachers.

# Table of Contents

Unit 1: Problem Solving

Unit 2: Introduction to Programming

Unit 3: Data Types and Operators

Unit 4: Control Structures

Unit 5: Arrays

Unit 6: Functions

Unit 7: File Handling

Unit 8: Emerging Technologies

DM

# Unit 1: Problem Solving

## Learning Objectives

• Define a problem and the problem-solving cycle.

• Draw flowcharts using standard symbols.

• Write algorithms in pseudocode.

• Test algorithms with dry runs.

## Key Concepts

• Problem-solving steps: understand, plan, execute, test

• Flowchart symbols: start/stop, input/output, process, decision

• Pseudocode conventions

• Dry run: tracing with sample data

• Selection and repetition in flowcharts

## Teaching Methodology (Step by Step)

1. Solve a daily problem (making tea) as an algorithm.
2. Draw flowchart symbols; students guess each meaning.
3. Convert a flowchart to pseudocode together.
4. Dry-run a summation algorithm on the board.

## Classroom Activities

• Draw flowcharts for: sum of N numbers, largest of three.

• Dry-run given algorithms and find outputs.

## Assessment Questions

1. Name the standard flowchart symbols.
2. Write pseudocode to find the sum of two numbers.
3. What is a dry run?
4. Draw a flowchart to check even or odd.

## Homework

• Draw five flowcharts in the notebook.

• Write pseudocode for three given problems.

## Time Allocation

5 periods of 40 minutes each

# Unit 2: Introduction to Programming

## Learning Objectives

• Define programming and programming languages.
• Classify languages: low-level vs high-level.
• Explain the role of compilers and interpreters.
• Write the structure of a simple C program.

## Key Concepts

• Program and programming
• Machine, assembly, high-level languages
• Compiler vs interpreter
• C program structure: headers, main(), statements
• Steps: edit, compile, run, debug

## Teaching Methodology (Step by Step)

1. Compare giving instructions to a human vs a computer.
2. Show the language-level ladder with examples.
3. Demonstrate compile vs interpret with an analogy.
4. Walk through a first C program line by line.

## Classroom Activities

• Classify ten languages as low or high level.
• Write and label the parts of a simple C program.

## Assessment Questions

1. Define a program.
2. What is the difference between a compiler and an interpreter?
3. Name two high-level languages.
4. Write the basic structure of a C program.

## Homework

• Write notes on language classification.
• Label the parts of three given C programs.

## Time Allocation

5 periods of 40 minutes each

# Unit 3: Data Types and Operators

## Learning Objectives

• Name the basic C data types.

• Declare variables and constants correctly.

• Use arithmetic, relational and logical operators.

• Write simple input/output statements.

## Key Concepts

• int, float, char, double

• Variable naming rules

• Constants: #define and const

• Operators: arithmetic, relational, logical, assignment

• printf() and scanf() basics

## Teaching Methodology (Step by Step)

1. Use labelled boxes as variables holding values.
2. Demonstrate each operator with quick examples.
3. Show common scanf mistakes and how to avoid them.
4. Trace small programs predicting their output.

## Classroom Activities

• Declare variables for five real-world values.

• Predict outputs of given code snippets.

## Assessment Questions

1. Name the four basic C data types.
2. Write two rules for variable names.
3. What does scanf() do?
4. Write the output of: printf("%d", 7 % 3);

## Homework

• Complete the exercise on operators.

• Write five small programs using printf/scanf.

## Time Allocation

5 periods of 40 minutes each

DM

# Unit 4: Control Structures

## Learning Objectives

• Explain sequence, selection and repetition.

• Write if, if-else and nested-if statements.

• Write switch statements.

• Write for, while and do-while loops.

## Key Concepts

• if, if-else, else-if ladder

• switch-case with break

• for loop: initialization, condition, increment

• while vs do-while

• break and continue

## Teaching Methodology (Step by Step)

1. Act out if-else with classroom decisions.
2. Trace an if-else program step by step.
3. Compare the three loops with the same counting task.
4. Show infinite loops and how to avoid them.

## Classroom Activities

• Write programs: grade finder, table printer.

• Convert a for loop into a while loop.

## Assessment Questions

1. Write the syntax of the if-else statement.
2. When is switch preferred over if-else?
3. Write a for loop to print 1 to 10.
4. What is the difference between while and do-while?

## Homework

• Write six programs using selection statements.

• Write six programs using loops.

## Time Allocation

6 periods of 40 minutes each

DM

# Unit 5: Arrays

## Learning Objectives

• Define an array and its advantages.

• Declare and initialize one-dimensional arrays.

• Process arrays with loops.

• Find sum, maximum and average of array elements.

## Key Concepts

• Array: collection of same-type elements

• Declaration: int marks[10];

• Indexing from 0

• Initialization methods

• Traversing with loops

## Teaching Methodology (Step by Step)

1. Use egg-tray analogy: one name, many compartments.
2. Show the off-by-one error with a live example.
3. Write a sum-of-marks program together.
4. Extend it to find maximum and average.

## Classroom Activities

• Write programs: reverse an array, search a value.

• Trace array programs predicting outputs.

## Assessment Questions

1. Define an array.
2. What is the index of the first element?
3. Declare an integer array of size 20.
4. Write a loop to input 10 numbers into an array.

## Homework

• Write five array programs.

• Trace three given array programs.

## Time Allocation

5 periods of 40 minutes each

DM

# Unit 6: Functions

## Learning Objectives

• Define a function and its benefits.

• Declare, define and call functions.

• Distinguish parameters and arguments.

• Explain local and global variables.

## Key Concepts

• Function: reusable block of code

• Declaration (prototype), definition, call

• Parameters vs arguments

• Return values; void functions

• Local vs global scope

## Teaching Methodology (Step by Step)

1. Compare functions to kitchen appliances: input, job, output.
2. Write a greet() function; call it from main().
3. Show parameter passing with an addition function.
4. Demonstrate local vs global with a variable clash.

## Classroom Activities

• Write functions: factorial, prime check, power.

• Convert repeated code into a function.

## Assessment Questions

1. Define a function.
2. What is the difference between a parameter and an argument?
3. Write a function to add two integers.
4. What is a local variable?

## Homework

• Write five function-based programs.

• Write notes on function components.

## Time Allocation

5 periods of 40 minutes each

# Unit 7: File Handling

## Learning Objectives

• Explain why files are needed.

• Open and close files in C.

• Write data to and read data from files.

• Explain text vs binary files.

## Key Concepts

• Need for permanent storage

• FILE pointer; fopen() modes (r, w, a)

• fprintf(), fscanf(), fgetc(), fputc()

• fclose() and why it matters

• Text vs binary files

## Teaching Methodology (Step by Step)

1. Ask where game scores are saved: the file idea.
2. Demonstrate writing names to a file and reading them back.
3. Explain each fopen mode with what happens to old data.
4. Show a program that appends to a file.

## Classroom Activities

• Write a program to save student names to a file.

• Write a program to count lines in a text file.

## Assessment Questions

1. Why do we use files?
2. Write the fopen modes and their meanings.
3. Which function writes formatted data to a file?
4. Why must we close a file?

## Homework

• Write three file-handling programs.

• Write notes on text vs binary files.

## Time Allocation

4 periods of 40 minutes each

# Unit 8: Emerging Technologies

## Learning Objectives

- Define artificial intelligence with examples.
- Explain the Internet of Things simply.
- Describe cloud computing and its services.
- Discuss the impact of technology on society.

## Key Concepts

- AI: machines doing smart tasks; examples
- IoT: connected devices (smart home)
- Cloud: storage and software online
- Big data idea in simple words
- Opportunities and challenges (jobs, privacy)

## Teaching Methodology (Step by Step)

1. Demo or discuss an AI chatbot's abilities.
2. List smart devices students use: the IoT web.
3. Compare saving files on USB vs cloud.
4. Debate: will AI take jobs or create them?

## Classroom Activities

- List ten AI uses in daily life.
- Design a dream smart-home with five IoT devices.

## Assessment Questions

1. Define artificial intelligence.
2. What is the Internet of Things? Give one example.
3. Name two advantages of cloud computing.
4. Write one opportunity and one challenge of AI.

## Homework

- Write a paragraph on AI in education.
- Make a chart of emerging technologies.

## Time Allocation

4 periods of 40 minutes each

DM